

Reliable Queue Workers

A project-authored field guide to dependable background work.

The examples use a small queue, explicit ownership, and observable state.

- Use a stable job identifier for every unit of work.
- Make handlers safe to run again after a process loss.
- Record the reason for every terminal outcome.

1. Queue Contracts

A queue contract names the work, its owner, and the acknowledgement boundary.

The worker may claim a job, but it must not claim success before durable completion.

- Payloads are versioned before they enter the queue.
- A lease has a visible expiry and a renewal rule.
- Acknowledgement follows the durable write.

2. Idempotent Jobs

Idempotency turns a replay from a duplicate side effect into a safe observation.

Persist the job key with the effect so the check and write share one transaction.

```
def handle(job, store):  
    if store.was_applied(job.key):  
        return "already-applied"  
    store.apply(job.key, job.payload)  
    return "applied"
```

3. Retries and Backoff

Retries are bounded observations of a failure, not a promise that work will eventually succeed.

- Classify failures before deciding whether a retry is safe.
- Increase the delay between attempts without hiding the attempt count.
- Send exhausted work to an explicit review path.

```
attempt=2
delay=$((attempt * 5))
printf 'retry in %s seconds\n' "$delay"
```

4. Visibility and Heartbeats

A worker should make ownership and progress visible without logging private payloads.

Heartbeat records describe the lease, not the contents of the job.

- Measure time since the last accepted heartbeat.
- Expose queue depth and age as raw operational facts.
- Keep provider and host details in bounded receipts.

5. Shutdown and Recovery

Graceful shutdown stops new claims, finishes safe work, and leaves open leases visible.

Recovery must distinguish a closed result from a process that disappeared mid-attempt.

```
def stop(worker):  
    worker.stop_claiming()  
    worker.drain_safe_work()  
    worker.record_open_leases()
```

- Never convert an interrupted attempt into a fabricated success.

6. Testing Worker Behavior

Tests should exercise the boundary between queue state and application state.

- Replay the same job key and verify one durable effect.
- Interrupt after claim and collect the open attempt.
- Reject malformed payloads before a handler runs.

```
python -m pytest tests/test_worker.py -q
```

```
python -m compileall worker.py
```

7. Operational Checklist

Use this short checklist before enabling a new worker in a local environment.

- Confirm the queue contract and idempotency key are documented.
- Confirm terminal receipts cannot be overwritten.
- Confirm failures remain distinguishable from incomplete work.

```
./worker --check-contract
```

```
./worker --print-lease-policy
```